

Cinc Cèntims

TALLER DE NOUS USOS DE LA INFORMÀTICA

Mario VILAR

13 de gener de 2025



UNIVERSITAT_{DE}
BARCELONA

if $n = 0$
if $n = 1$
if $n \geq 2$

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ f(n-1) + f(n-2) & \text{if } n \geq 2 \end{cases}$$

ÍNDIX

1	Introducció	3
2	Recomanadors	6
3	Classificació	9
4	El problema dels ítems freqüents	11
5	Hipòtesis, inferències i A-B testing	14
6	Explotació i exploració	15
6.1	n-armed bandits	15
6.2	Mètodes acció-valor	16
6.3	Incertesa: UCBi i Thompson Sampling	18
7	El teorema de Bayes	20
8	Models de dades i Bayes	23
8.1	Model probabilístic	23
8.2	Model de bigrames	26
8.3	MNIST classification with NN	27
9	Aspectes ètics	32
9.1	Justícia	33
9.2	Mesures d'equitat	35
A	Notebooks	37
A.1	Single-layer perceptrons	37
A.2	Multi-Layer Perceptron	40
A.3	Multiclass classification	40
A.4	Deep Learning and backpropagation	41

comptar quantes vegades surt una diferència més gran que la observada i assignar aquesta probabilitat al valor observat.

Definició 5.6 (Bootstrap). El mètode de bootstrap ens permet crear una aproximació robusta de la distribució mostral d'aquests paràmetres (inclosa la mitjana) a partir d'aplicar un mostreig aleatori amb reemplaçament. Donat un conjunt d' N elements, el mostreig aleatori amb reemplaçament consisteix en:

1. Assignar un nombre enter a cada element.
2. Seleccionar N elements del conjunt (alguns d'ells possiblement repetits) mitjançant la generació de nombres aleatoris de l'interval d'enters $(1, \dots, N)$.

Conclusions

- L'estadística ens ajuda a quantificar l'incertesa d'un resultat, però el significat d'un resultat no depèn de les dades ni de la validació dels resultats, sinó de l'analista.
- Sempre cal ser escèptic i no fer proposicions massa agosarades sobre els resultats. L'única forma d'augmentar la certesa sobre algun efecte és repetir l'experiment moltes vegades.

6

EXPLOTACIÓ I EXPLORACIÓ

Exploitar és triar l'acció que *creiem* que dona més recompenses, mentre que *explorar* és millorar les nostres estimacions sobre el valor de les altres recompenses.

6.1 | n-ARMED BANDITS

Suposem que cada vegada que arriba un usuari al nostre web site podem oferir-li una opció que triarem entre les n opcions que tenim disponibles en aquell moment (anuncis, circuits de vacances, itineraris d'avió, etc.). Un cop oferta l'opció, i en funció de l'acció de l'usuari, rebem una recompensa que es pot representar numèricament (que pot ser simplement si ha estat seleccionada o no, el preu que paga o no l'usuari per aquella opció, etc.).

Suposem que la recompensa per cada opció es pot modelar com una funció estacionària de probabilitats (és a dir, la recompensa és una variable aleatòria, no és la mateixa per totes les ofertes, com és el cas d'una màquina *tragaperras*).

Si el nostre objectiu és maximitzar la recompensa total al cap del temps i coneguéssim la mitjana de la recompensa real de cada acció (coneixement que en general no és possible tenir), triaríem oferir sempre la millor acció. Aquest seria l'algorisme òptim, A_{opt} .

Definició 6.1 (Jugada). Un objectiu interessant i més realista en aquest escenari és trobar un algorisme que maximitzi la *recompensa total esperada* sobre un número d'accions. A cada una de les accions l'anomenem *jugada*.

Definició 6.2 (Regret). Volem minimitzar una mesura de penediment (*regret*), el total de recompenses que perdem pel fet de no realitzar sempre la millor acció, després de T accions:

$$R(T) = \text{Guanys}_{A_{opt}}(T) - \text{Guanys}_A(T)$$

Aquesta formulació es coneix com el problema dels n -armed bandits, o de les màquines escurabutxaques amb actuadors, per analogia amb les màquines escurabutxaques o one-armed bandit (que tenen només un actuator i en les que només hem de decidir si volem continuar jugant).

Definició 6.3 (Valor). En el nostre cas, cada acció té una recompensa esperada o recompensa mitja donada l'acció que es selecciona. Aquest concepte s'anomena *valor* de l'acció.

Observació 6.4.

- Si sabéssim el valor de cada acció la solució del problema seria trivial i consistiria en seleccionar sempre l'acció amb més valor. En el nostre cas no saben el valor real de cada acció, però en podem tenir estimacions a mesura que anem jugant!
- La tria entre explotar o explorar depèn d'una forma complexa de:
 - Valors específics de les estimacions.
 - Incerteses sobre aquestes estimacions.
 - Nombre de jugades que ens queden per fer.

6.2 | MÈTODES ACCIÓ-VALOR

Primer hem de veure com estimar els valors de les accions i com usar aquestes estimacions per decidir quina acció seleccionem.

1. Anomenem $Q^*(a)$ al valor real de l'acció a , que és desconegut. Recordem que aquest valor és la mitja de les recompenses rebudes quan seleccionem aquesta acció.
2. Anomenem $Q_t(a)$ al seu valor estimat després de t jugades.

Definició 6.5 (Mètode de la mostra promitjada). Si suposem que fins a la jugada t l'acció a ha estat seleccionada k_a vegades i hem obtingut unes recompenses $r_1 + \dots + r_t$, la forma natural d'estimar $Q_t(a)$ és:

$$Q_t(a) = \frac{r_1 + \dots + r_{k_a}}{k_a}. \quad (6.1)$$

Definim $Q_t(a)$ amb valor inicial $Q_0(a) = 0$. Sabem que $Q_t(a) \xrightarrow{k_a \rightarrow \infty} Q^*(a)$ per la llei dels grans nombres. A aquest mètode l'anomenem el mètode de la mostra promitjada atès que per estimar els valors de les accions fem una simple mitja de les recompenses relacionades.

¹ El problema amb aquesta fórmula és que els requeriments de memòria i càlcul creixen amb el temps, atès que hem de recalculer aquesta expressió a cada pas per cada acció a i hem d'anar guardant la llista de valors $(r_1, \dots, r_k)$ per a cada acció, de manera que anirà creixent. Més endavant en parlarem.

Definició 6.6 (Mètode voraç).

1. Fer t jugades per estimar el valor estimat de cada acció,
2. A partir d'aleshores, triar l'acció amb el valor estimat més alt, o dit d'una altra manera, seleccionar a partir de la jugada $t + 1$ l'acció voraç a^* : $Q_t(a^*) = \max_a Q_t(a)$.

Aquest mètode explota el coneixement que hem adquirit fins a la jugada per maximitzar la recompensa, però perd molt valor durant el període fins a realitzar el total de les accions.

Definició 6.7 (Mètodes ε -voraços). Una possible alternativa és actuar de forma voraç la majoria de vegades però de tant en tant, amb una petita probabilitat ε , seleccionar una altra acció de forma uniformement aleatòria, independentment de les estimacions d'acció-valor.

Observació 6.8.

1. Una propietat d'aquests mètodes (que no compleix el mètode voraç, 6.6) és que a mesura que augmenta el nombre de jugades cada una de les accions serà mostrejada un nombre infinit de vegades, garantint que $k_a \xrightarrow{t \rightarrow \infty} \infty$ per totes les a i, per tant, assegurant que $Q_t(a)$ convergeix a $Q^*(a)$.
2. Tot i que els mètodes ε -voraços són simples i efectius per balancejar l'exploració i l'explotació, podem tenir un petit defecte: quan exploren trien de forma aleatòria. És a dir, entre totes les accions per igual.

Solució: mètode softmax.

Definició 6.9 (Mètode softmax). Triar les accions segons una funció que les distribueixi en funció del seu valor estimat. El mètode escull una acció a a la jugada t segons aquesta distribució de probabilitats:

$$\frac{e^{\frac{Q_t(a)}{\tau}}}{\sum_{b=1}^n e^{\frac{Q_t(b)}{\tau}}}.$$

τ és un paràmetre positiu anomenat *temperatura*.

- Temperatures altes ($\tau \rightarrow \infty$) fan que les accions siguin equiprobables (distribució uniforme) i per tant totes tenen la mateixa probabilitat de ser explorades.
- Quan $\tau \rightarrow 0$ tenim el cas voraç pur i només explotem la que creiem més valuosa.

Intentem definir (6.1) de manera recursiva per intentar estalviar-nos espai a memòria. Sigui Q_k la mitja de les seves primeres recompenses. Llavors, quan obtenim la recompensa $k + 1$, r_{k+1} , podem calcular la mitja de les recompenses com:

$$Q_{k+1} = Q_k + \frac{1}{k+1}(r_{k+1} - Q_k),$$

ja que:

$$\begin{aligned} Q_{k+1} &= \frac{1}{k+1} \sum_{i=1}^{k+1} r_i = \frac{1}{k+1} \left(r_{k+1} + \sum_{i=1}^k r_i \right) = \frac{1}{k+1} (r_{k+1} + kQ_k) \\ &= \frac{1}{k+1} (r_{k+1} + (k+1)Q_k - Q_k) = Q_k + \frac{1}{k+1} (r_{k+1} - Q_k) \end{aligned}$$

Aquesta expressió serveix fins i tot per a $k = 0$ i dona $Q_1 = r_1$ per un Q_0 arbitrari.

Observació 6.10. Aquesta formulació incremental segueix una forma general interessant:

$$\text{nova estimació} \leftarrow \text{estimació anterior} + \text{mida del pas} \cdot (\text{valor actual} - \text{estimació anterior}) .$$

- La part $\text{valor actual} - \text{estimació anterior}$ es pot interpretar com l'error de l'estimació i per tant el mètode el que fa és fer un pas en la direcció del valor actual (en el nostre cas, el valor actual és la recompensa r_{k+1}).
- Tal i com ho hem formulat, la mida del pas, que es pot escriure com $\alpha_k(a)$, canvia a cada pas: quan processem la recompensa k de l'acció a el mètode usa una mida $\frac{1}{k}$. Això ho fa per mantenir el pes de totes les recompenses igualat quan fa la mitjana.
- *Cas dels problemes no estacionaris:* Si la distribució de recompenses va variant amb el temps (cas no estacionari) no té sentit estimar les recompenses com ho fem, amb el mateix pes: hem de donar més pes a les més recents.

Una manera d'implementar el cas dels problemes no estacionaris és usar una mida de pas constant a l'equació. Per exemple, $Q_{k+1} = Q_k + \alpha(r_{k+1} - Q_k)$. En aquest cas, decidim que la mida del pas α ($0 < \alpha \leq 1$) és constant. Això resulta en una fórmula on Q_k és una mitja ponderada de les recompenses passades i de l'estimació inicial Q_0 :

$$\begin{aligned} Q_k &= Q_{k-1} + \alpha(r_k - Q_{k-1}) = \alpha r_k + (1 - \alpha)Q_{k-1} \\ &= \alpha r_k + (1 - \alpha)r_{k-1} + (1 - \alpha)^2 \alpha r_{k-2} + \dots + (1 - \alpha)^{k-1} \alpha r_1 + (1 - \alpha)^k \alpha Q_0 \\ &= (1 - \alpha)^k Q_0 + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} r_i. \end{aligned}$$

Això és una mitja ponderada perquè la suma dels pesos és:

$$(1 - \alpha)^k + \sum_{i=1}^k \alpha (1 - \alpha)^{k-i} = 1.$$

El pes de la recompensa r_i , $\alpha(1 - \alpha)^{k-i}$, va decreixent exponencialment en funció a mesura que el nombre de recompenses augmenta (que és el que volíem).

6.3 | INCERTESA: UCB I THOMPSON SAMPLING

Suposem ara que tenim k accions possibles i les seves recompenses estimades després de t accions, $Q_t(a)$. Cada una d'aquestes k accions a ha estat estimada fent una mitjana formada per diversos valors. Estadísticament, podríem calcular la incertesa de cada una d'aquestes mitjanes.

Exemple 6.11. Suposem tres valors estimats μ_a i les seves incerteses u_a : 5 ± 1 , 4.5 ± 2 i 1 ± 3 . Quina acció es podria triar? En règim d'explotació pura, la primera, perquè és la que té el valor estimat més gran. En règim d'exploració pura, la tercera, que és la més incerta.

L'interessant és trobar una via intermitja que no ens obligui a triar el règim i vagi variant de forma automàtica entre els dos.

Definició 6.12 (UCB_i). L'algorisme UCB_i (Upper Confidence Bound) fa la següent tria: escollirem l'acció a^* tal que $a^* = \operatorname{argmax}_a (\mu_a + u_a)$. Per calcular l'incertesa s'usa una expressió basada en la cota de Hoeffding:

$$Q_t(a) \pm \sqrt{\frac{2 \ln t}{t_a}},$$

on t és el nombre d'accions totals que hem fet fins el moment i t_a és el nombre de vegades que s'ha mostregjat l'acció a :

- Cada cop que seleccionem l'acció a , t_a es fa més gran i l'incertesa disminueix.
- Cada cop que seleccionem una acció que no és a , es fa més gran, però amb un ordre logarítmic (més a poc a poc que el lineal o el quadràtic).

Observació 6.13. En la teoria de la probabilitat, la cota de Hoeffding proporciona un límit superior a la probabilitat que la suma de les variables aleatòries independents es desviï del seu valor esperat en més d'una certa quantitat.

Definició 6.14 (Thomson Sampling). El mètode Thomson Sampling modela la incertesa mitjançant la construcció d'una distribució de probabilitat a partir de recompenses històriques i després, a cada pas, mostreja aquesta distribució per triar les accions. En el cas senzill en què les recompenses són binàries (per exemple, sí vs no), s'utilitza una distribució Beta.

Definició 6.15 (Distribució beta). En teoria de la probabilitat i estadística, la distribució beta és una família de distribucions de probabilitat contínues definides en l'interval $[0, 1]$, parametritzades per dos paràmetres de forma, denotats α i β , que apareixen com a exponents de la variable aleatòria i controlen la forma de la distribució. La PDF (no fa falta aprendre-se-la) és:

$$f(x) = \frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha, \beta)}, \text{ on } B(\alpha, \beta) = \frac{\Gamma(\alpha)\Gamma(\beta)}{\Gamma(\alpha + \beta)}. \quad 0 \leq x \leq 1, \alpha, \beta > 0.$$

L'esperança d'aquesta distribució és $E(X) = \frac{\alpha}{\alpha + \beta}$. Aquesta esperança (o valor mig) representa el nombre de «recompenses positives» dividit entre «recompenses positives i no positives». Per seleccionar una acció, prenem una mostra de cada opció segons els seus paràmetres i escollim l'acció amb el valor màxim.

1. Abans de començar, inicialitzarem els paràmetres de la funció Beta a $\alpha = \beta = 1$, que correspon a una distribució uniforme entre 0 i 1 (i.e. $B(1, 1) \sim U(0, 1)$).
2. Un altre avantatge de la funció Beta és la simplicitat de la seva actualització després de fer una acció. Quan obtenim una recompensa després d'una acció a (que en el cas binari és $r_{t+1} = 1$) o no (que en el cas binari és $r_{t+1} = 0$), els paràmetres de la funció Beta de l'acció es poden actualitzar així:

$$\alpha_{r_{t+1}}(a) = \alpha_{r_t}(a) + r_{t+1}\beta_{r_{t+1}}(a) = \beta_{r_t}(a) + 1 - r_{t+1}.$$

3. A mesura que recollim més dades, α i β augmenten. Com a resultat, la distribució Beta es fa més estreta i obtenim més certesa en la nostra estimació del valor. *Curtosi*: Amb una distribució més estreta, els nostres valors mostrejats estaran més a prop de la mitjana, reduint així l'exploració i augmentant l'exploació.

Algoritme 6.16. *Suposant que cada possible acció a_i està representada per $\text{Beta}(a_i)$, a cada pas de l'algorisme fem:*

1. *Generem, per cada a_i , un valor aleatori v_{a_i} segons la funció $\text{Beta}(a_i)$.*
2. *Executem l'acció i tal que té v_{a_i} el valor màxim d'entre els generats.*
3. *Recalculem les $\text{Beta}(a_i)$ per totes les possibles accions.*

En la majoria de casos, aquesta estratègia és millor que la UCB_L.

Exemple 6.17 (Aplicacions).

- Assaig clínic per determinar el millor tractament d'una malaltia.
- Escollir el millor servidor en una granja d'ordinadors per fer una tasca.
- Portafolis financers (inversions).
- Recomanació, quan el nombre d'ítems no és molt gran. En aquest context també podem aplicar una variació dels mètodes vistos, contextual bandits, que incorpora informació sobre els ítems.

Conclusions

- L'ús de la incertesa produeix millor mètodes: Els mètodes UCB i Thomson Sampling són en general millor que els mètodes ϵ -greedy.
- Quan la informació sobre la recompensa es retarda, Thompson Sampling supera UCB. Aquesta retroalimentació retardada, on les interaccions usuari-element no es processen immediatament, és habitual per a la majoria de sistemes del món real (com per exemple les màquines *tragaperras*).
- En aquesta situació, com que la UCB selecciona les armes de manera determinista, tria la mateixa acció fins que s'incorpora una nova retroalimentació. En canvi, com que Thompson Sampling tria les accions de manera estocàstica mitjançant el mostreig de la distribució posterior, aleatoritza les accions fins i tot sense recompenses actualitzades.

EL TEOREMA DE BAYES

Anomenem $P(A)$ probabilitat *marginal*, $P(A, B) \equiv P(A \cap B)$ probabilitat *conjunta* (a la vegada d' A i B) i $P(A|B)$ la probabilitat *condicional* (probabilitat d' A coneixent B).

Teorema 7.1 (de Bayes). *Sigui un conjunt de dades representat per característiques $(A, B, C, \dots)$. Tenim:*

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}.$$

Demostració. Sabem que $P(B|A)P(A) = P(A \cap B)$ i $P(A|B)P(B) = P(A \cap B)$. Combinant-les, $P(A)P(B|A) = P(B)P(A|B)$. És útil sempre i quan el terme de la dreta de la igualtat sigui més fàcil de calcular que $P(A|B)$. ■

Definició 7.2 (Interpretació diacrònica). Donades unes dades D (o *evidències*) i un conjunt de possibles *hipòtesis* sobre aquestes dades H , voldríem un mètode objectiu per modificar la nostra hipòtesi en funció de les dades. Llavors, podem definir:

1. $P(H)$ o la probabilitat a priori de cada hipòtesi, ABANS de veure les dades.
2. $P(H|D)$ o la probabilitat de cada hipòtesi DESPRÉS de veure les dades.

Expressant-ho amb la fórmula de Bayes ens queda:

$$P(H|D) = \frac{P(D|H)P(H)}{P(D)}.$$

$P(D|H)$, tal com veurem, és normalment fàcil de calcular. En canvi, l'única forma de calcular $P(H|D)$, que és el que realment ens interessa, és amb la fórmula de Bayes!

Definició 7.3 (Interpretació freqüentista). L'estadística bayesiana interpreta les probabilitats com a mesures de *credibilitat* (la *confiança* que tenim) en un esdeveniment o hipòtesi, no com la freqüència d'esdeveniments a llarg termini. Les diferents creences no s'interpreten com a errors sinó com a diferents estats de coneixement sobre un esdeveniment per part de diferents observadors. La fórmula s'interpreta com una actualització de la creença després d'observar les dades.

Exemple 7.4. Volem assignar una probabilitat al fet que Bin Laden estigui mort. En aquest cas, la noció freqüentista de sèrie d'assaigs no està ben definida: no hi ha sèries d'assaigs idèntics (basats en esdeveniments repetibles) a considerar. Podem concloure que la noció clàssica de probabilitat no s'aplica a aquestes situacions. El bayesianisme defineix la probabilitat d'una altra manera: el grau de creença que es produirà un esdeveniment.

Per a un freqüentista no hi ha probabilitat d'aquest esdeveniment perquè no hi ha proves possibles (Bin Laden ha mort o no, no és una qüestió de probabilitat). Un bayesià assignaria una probabilitat a aquest esdeveniment en funció del seu estat de coneixement. L'estat del coneixement canvia quan es disposa de nova informació.

Definició 7.5 (Versemblança). La versemblança $P(E|H)$ representa com de bé una determinada hipòtesi explica unes dades. Si les dades són independents, es calcula com $\prod_i p_H(x_i)$, on p_H és la funció de distribució o llei de les dades *sota la hipòtesi* H .

Teorema 7.6 (de les probabilitats totals). Si hi ha n hipòtesis que són mútuament exclusives i col·lectivament exhaustives², podem calcular la probabilitat de l'esdeveniment E , $P(E)$, com:

$$P(E) = \sum_{i=1}^n P(H_i)P(E|H_i) = P(H_1)P(E|H_1) + \cdots + P(H_n)P(E|H_n).$$

² Manera completament pedant de dir que $\{H_1, \dots, H_n\}$ és una partició de l'espai mostral Ω .

En general, tal com veurem, $P(H|E)$, $P(H)$, $P(E|H)$, $P(E)$ seran funcions.

Exemple 7.7. Suposem que fem una prova mèdica per diagnosticar una malaltia a una població amb les següents característiques.

- Sabem que $P(\{\text{sí}\}) = 0.05$. Això s'anomena *prevalença* d'una malaltia.
- A cada individu se li fa una prova amb precisió $P(+|\{\text{sí}\}) = P(-|\{\text{no}\}) = 0.9$. Això és fàcil de calcular amb una prova clínica.

Volem calcular $P(\{\text{sí}\} | +)$. Podem utilitzar la regla de Bayes per calcular aquesta probabilitat a posteriori:

$$P(\{\text{sí}\} | +) = \frac{P(+|\{\text{sí}\}) \cdot P(\{\text{sí}\})}{P(+)} = \frac{P(+|\{\text{sí}\}) \cdot P(\{\text{sí}\})}{P(+|\{\text{sí}\}) \cdot P(\{\text{sí}\}) + P(+|\{\text{no}\}) \cdot P(\{\text{no}\})}.$$

Fent els càlculs, $P(\{\text{sí}\} | +) = \frac{0.05 \cdot 0.9}{0.05 \cdot 0.9 + 0.95 \cdot 0.1} \approx 0.32$.

Exemple 7.8 (Monty Hall). Definició d'esdeveniments: T_A, T_B, T_C triem portes A, B, C ; C_A, C_B, C_C cotxe és a A, B, C ; P_A, P_B, P_C presentador obre porta A, B, C . E és èxit, guanyem el cotxe. L'èxit es produirà sempre i quan T_i i C_i coincideixin. Comptem casos.

1. Si el presentador no fes res,

$$E = (T_A \cap C_A) \cup (T_B \cap C_B) \cup (T_C \cap C_C) \implies P(E) = 3 \cdot \left(\frac{1}{3} \cdot \frac{1}{3} \right) = \frac{1}{3}.$$

2. Si no canviem porta:

$$\begin{aligned} E &= [(T_A \cap C_A \cap P_B) \cup (T_A \cap C_A \cap P_C)] \cup [(T_B \cap C_B \cap P_A) \cup (T_B \cap C_B \cap P_C)] \\ &\cup [(T_C \cap C_C \cap P_A) \cup (T_C \cap C_C \cap P_B)] \implies P(E) = 3 \cdot \left(2 \cdot \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{2} \right) = \frac{1}{3}. \end{aligned}$$

3. Si canviem la porta:

$$\begin{aligned} E &= [(T_A \cap C_C \cap P_B) \cup (T_A \cap C_B \cap P_C)] \cup [(T_B \cap C_C \cap P_A) \cup (T_B \cap C_A \cap P_C)] \\ &\cup [(T_C \cap C_B \cap P_A) \cup (T_C \cap C_A \cap P_B)] \implies P(E) = 3 \cdot \left(2 \cdot \frac{1}{3} \cdot \frac{1}{3} \cdot \frac{1}{1} \right) = \frac{2}{3}. \end{aligned}$$

La clau està en el presentador, que obrirà una porta diferent a la del cotxe.

Exemple 7.9 (El problema de la locomotora). Una companyia ferroviària numera les seves locomotores per ordre $2, \dots, N$. Un dia veus una locomotora amb el número 60. Pots estimar quantes locomotores té la companyia ferroviària? Per aplicar el raonament bayesià, podem dividir aquest problema en dos passos:

1. Què sabem sobre abans de veure les dades? $P(H)$.
2. Per a qualsevol valor de N , quina és la probabilitat de veure les dades (una locomotora amb el número 60)? $P(E|H)$.

Si hi ha N locomotores, possibilitat de veure qualsevol vagó en particular és d' $\frac{1}{N}$. Calculant $P(H)P(E|H)$ per a cada $H = \{\text{la locomotora té } N \text{ vagons}\}$ (posem $1 \leq N \leq 1000$, per exemple). Matemàticament podríem demostrar que $2 \cdot X - 1$ ens donaria un estimador sense biaix d' N i 60 seria l'EMV (estimador de màxima versemblança). Els bayesians proposen com alternativa calcular la hipòtesi que correspon al valor mitjà de la distribució posterior, que seria 333.

8

MODELS DE DADES I BAYES

Definició 8.1.

- Una col·lecció de textos s'anomena corpus. El corpus està format per una seqüència de tokens, paraules i signes de puntuació/blancs.
- Cada token *different* s'anomena tipus³.
- El conjunt de tots els tipus s'anomena vocabulari.
- Una seqüència d'un token és un unigrama, una de dos token és un bigrama i una de n tokens és un n -grama.

L'any 2006, Google va fer públic un conjunt de dades estadístiques extretes d'una sèrie de documents en llenguatge natural que contenen més de 1.000.000.000.000 paraules en anglès. Les dades resumeixen els documents comptant el nombre de vegades que hi apareix una determinada paraula i també les seqüències de dues, tres, quatre i cinc paraules concretes.

Notació 8.2.

- Direm que un cert tipus té una probabilitat P , com per exemple $P(\text{the}) = 0.022$, quan la freqüència del tipus al corpus sigui d'un $P \times 100\%$.
- Si W és una seqüència de tokens, W_3 representa el tercer token i representa la seqüència del primer al tercer.
- $P(W_i = \text{the} | W_{i-1} = \text{of})$ s'anomena la probabilitat condicional de *the*, donat que *of* és el token anterior.

8.1 | MODEL PROBABILÍSTIC

El problema serà segmentació de paraules, i la solució a partir dels models probabilístics.

Definició 8.3 (Model de llenguatge). En el nostre cas, l'anomenarem model de llenguatge i serà una distribució de probabilitats sobre tots els possibles strings que es puguin formar a partir dels tipus definits. Els paràmetres d'aquest models s'aprendran del corpus i ens serviran per assignar una probabilitat a cada possible solució.

- Definim un model probabilístic.

³ Per tant, la frase «Run, Lola, Run» té 5 tokens (la coma compta) però només 3 tipus.

2. Enumerem les solucions candidates. En el nostre cas, hem d'enumerar de forma eficient totes les possibles segmentacions de la nostra seqüència.
3. Escollim la solució amb màxima probabilitat. En el nostre cas, aplicarem el model de llenguatge a cada possible solució candidata.

En terminologia probabilística, el que volem s'escriu així:

$$\text{solució} = \underset{c \in \text{candidats}}{\operatorname{argmax}} P_M(c), \quad M \text{ és un model de llenguatge,}$$

calculat a partir del corpus. A partir d'ara, com que només considerarem un model, escriurem directament $P(c)$. La probabilitat d'una seqüència de paraules és pot escriure com:

$$P(W_{1:n}) = \prod_{k=1}^n P(W_k | W_{1:k-1}) = P(W_1) \cdot P(W_2 | W_1) \cdots P(W_n | W_{1:n-1}).$$

És evident que no hi ha prou dades per calcular aquestes probabilitats, atès que hauríem de calcular $P(W_k | W_{1:k-1})$ per cada possible seqüència $W_{1:k-1}$. Podem considerar el model simplificat que només pren en consideració els unigrames:

$$P(W_{1:n}) \approx \prod_{k=1}^n P(W_k).$$

Bàsicament el que fem és considerar que la probabilitat d'una lletra és independent a les anteriors lletres de la paraula.

Exemple 8.4. Per segmentar *lacasade*, hem de considerar com a solució candidata el conjunt $\{la, casa, de\}$ i calcular la seva probabilitat $P(la) \cdot P(casa) \cdot P(de)$. Si cap altre conjunt un producte més gran de probabilitats, llavors ja hem trobat la solució. Si cap altre conjunt dona un producte més gran de probabilitats, llavors ja hem trobat la solució.

1. Un string de n caràcters té 2^{n-1} segmentacions possibles, atès que hi ha $n - 1$ llocs possibles entre lletra i lletra.
2. Per tant, l'enumeració de tots els candidats possibles no és una opció i hem de buscar una opció més eficient.
3. Podem implementar una aproximació recursiva definida a partir de segmentació entre una primera paraula (la longitud de la qual està limitada per la longitud de la paraula més llarga del corpus) i la resta del text.
4. De tots els possibles candidats, la que té $P(\text{first}) \cdot P(\text{remaining})$ màxima és la millor.

```

1 @memo
2 def segment(text):
3     '''
4     Return a list of words that is the best segmentation of text.
```

```

5      '''
6      if not text:
7          return []
8      candidates = [[first]+segment(rem) for first, rem in split(text)]
9      return max(candidates, key=Pwords)
10 def splits(text, L=20):
11     '''
12     Return a list of all possible (first, rem) pairs, len(first)<=L.
13     '''
14     return [(text[:i+1], text[i+1:]) for i in range(min(len(text), L))]
15 def Pwords(words):
16     '''
17     The Naive Bayes probability of a sequence of words.
18     '''
19     return product([PW(w) for w in words])

```

1. product és una funció que multiplica una llista de nombres.
2. PW és una funció que retorna la probabilitat d'una paraula de la llista dels unigrames.
3. memo és un decorador de Python que manté en memòria els resultats de les crides anteriors d'una funció de manera que no s'han de recalculer si es tornen a necessitar.

```

1 def memo(f):
2     '''
3     Memoize function f.
4     '''
5     table = {}
6     def fmemo(*args):
7         if args not in table:
8             table[args] = f(*args)
9         return table[args]
10    fmemo.memo = table
11    return fmemo

```

1. Sense memo la crida a la funció segment amb un string de n caràcters implica 2^n crides recursives.
2. Amb memo ho transformem en un procés de programació dinàmica i només en fa n . Cada crida considera $O(L)$ segmentacions (on L és la longitud màxima d'una paraula), i per cada una avalua $O(n)$ probabilitats, per tant la complexitat és $O(n^2 L)$.
3. Respecte PW, accedeix al fitxer amb els estadístics (count) de cada paraula i estima la probabilitat com $\text{count}(\text{word})/N$, on N és el nombre d'elements del corpus.

Observació 8.5. Què fem si la paraula no és al diccionari (no podem retornar 0)? La paraula menys vista del diccionari té una probabilitat de $p(\frac{k}{N})$ per a cert k i, per tant, una paraula no vista hauria de tenir una probabilitat menor.

Una heurística que funciona és assignar una probabilitat inicial de $\frac{k}{N}$ però decrementem per un factor 10 per cada lletra de la paraula no vista (com més llarga, més improbable).

8.2 | MODEL DE BIGRAMES

Un dels errors que hi ha és que no ha segmentat certes paraules, com ara *sitdown*. El motiu és que el producte de les probabilitats de les dues paraules (que tenen probabilitats 0.003% i 0.04% respectivament) és lleugerament inferior que la de *sitdown*. Però si miréssim les probabilitats dels bigrames, veuríem que la probabilitat de *sitdown* és 100 vegades superior a la de *sitdown*!

Definició 8.6 (Model de bigrames). Per tant, aquest error es pot resoldre considerant un model de bigrames:

$$P(W_{1:n}) = \prod_{k=1, \dots, n} P(W_k | W_{k-1}).$$

Però la taula amb els valors de tots els bigrames no cap a la memòria!

Si un bigrama no apareix a la taula, llavors usem l'estimació dels unigrames.

```

1 def cPw(word, prev):
2     '''
3     The conditional probability P(word | previous-word).
4     '''
5     try:
6         return P2w[prev + ' ' + word] / float(Pw[prev])
7     except KeyError:
8         return Pw(word)
9 P2w = Pdist(datafile('count2w'), N)
```

La funció cPw no genera una distribució de probabilitats (pot sumar més que 1), tot i que a la pràctica funciona bé. Per aquest motiu s'anomena *stupid backoff*.

Observació 8.7. Quan apliquem la funció Pwords a una seqüència llarga de paraules podem tenir problemes d'underflow (intentem representar un nombre tan petit que a l'ordinador s'interpreta com a 0) per culpa del nombre de multiplicacions. Una solució senzilla és treballar amb els logaritmes de les probabilitats i fer una suma de logaritmes enlloc d'una multiplicació (i.e. $\log \prod_i x_i = \sum_i \log x_i$).

```

1 from math import log10
2 @memo
3 def segment2(text, prev='\langle S \rangle'):
4     '''
5     Return (log P(words), words), where words is the best segmentation.
6     '''
7     if not text:
```

```

8     return 0.0, []
9     candidates = [combine(log10(cPw(first, prev)), first, segment2(rem, first)) for
    ↪ first, rem in splits(text)]
10    return max(candidates)

```

8.3 | MNIST CLASSIFICATION WITH NN

The MNIST database is a good database for people who want to try learning techniques and pattern recognition methods on real-world data while spending minimal efforts on preprocessing and formatting. The database is also widely used for training and testing in the field of machine learning.

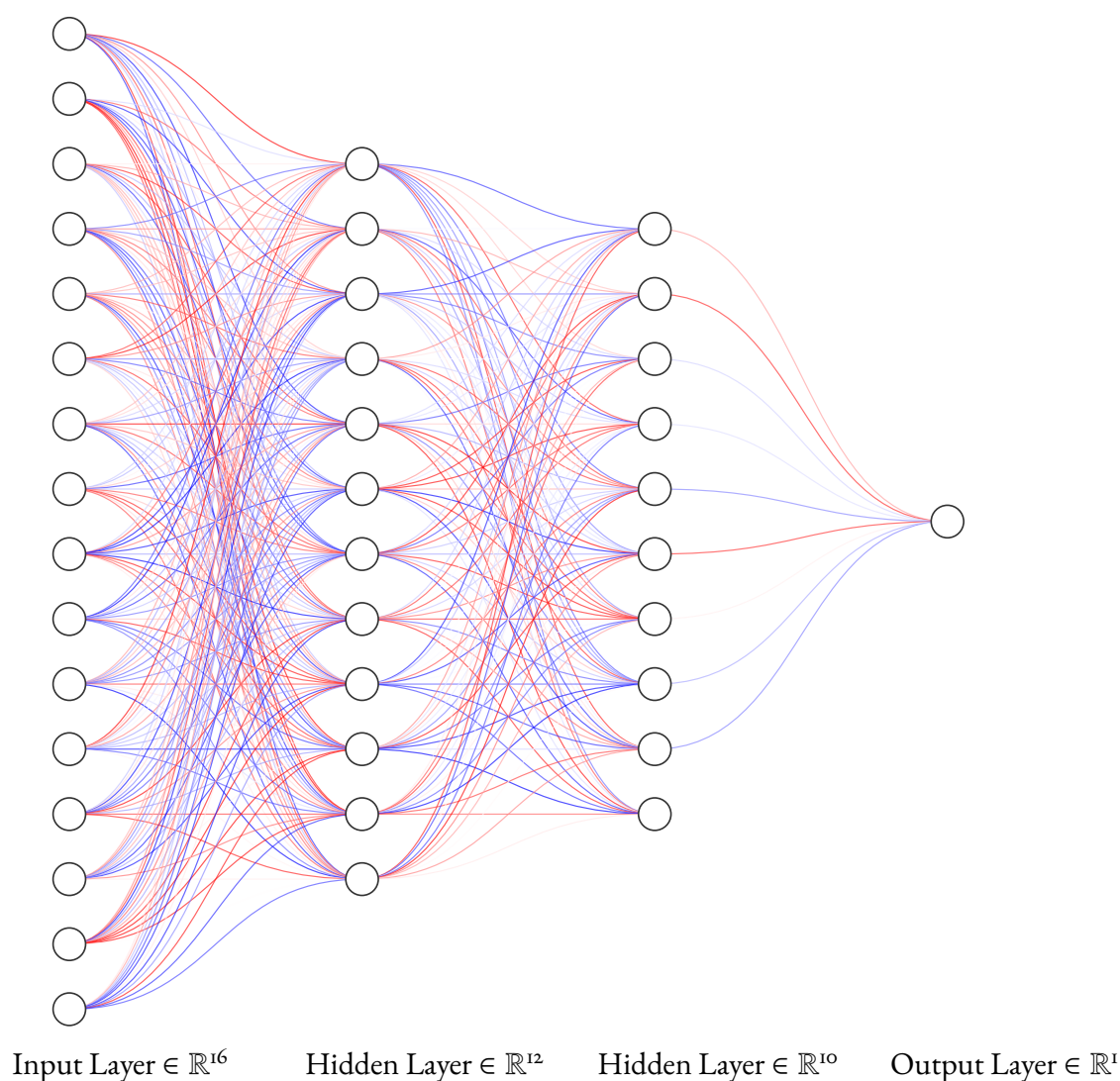


Figura 2: Neural Network. Si en la capa d'input tenim n nodes i en la primera capa oculta en tenim m , tindrem un total de $n \cdot m + m$ pesos.

8.8 (Images and Fully Connected (FC) neural networks).

- Cada píxel és una entrada per a la xarxa.
- Problema amb imatges grans: massa dimensions i paràmetres. Flattening operation. This function is used to get a copy of a given tensor collapsed into one dimension.

Exemple 8.9. Say we have a dataset of one-megapixel photographs we need to classify. This means that each input to the network has one million dimensions. Even an aggressive reduction to 1000 hidden dimensions would require a fully-connected layer characterized by $10^6 \cdot 10^3 = 10^9$ parameters. Moreover, learning a classifier by fitting so many parameters might require collecting an enormous dataset.

Convolutional neural networks (CNNs) are one creative way that machine learning has embraced for exploiting some of the known structure in natural images in order to minimize the number of parameters.

8.10 (Fully connected vs. convolutional layers).

- Fully connected networks have no notion of «nearby» and treat the relationship between *every input equally*.
- Convolutional layers process each local image region independently, using parameters shared across the whole image.
- Convolutional layers use *fewer parameters* than fully connected layers, exploit the spatial relationships between nearby pixels, and don't have to re-learn the interpretation of the pixels at every position.

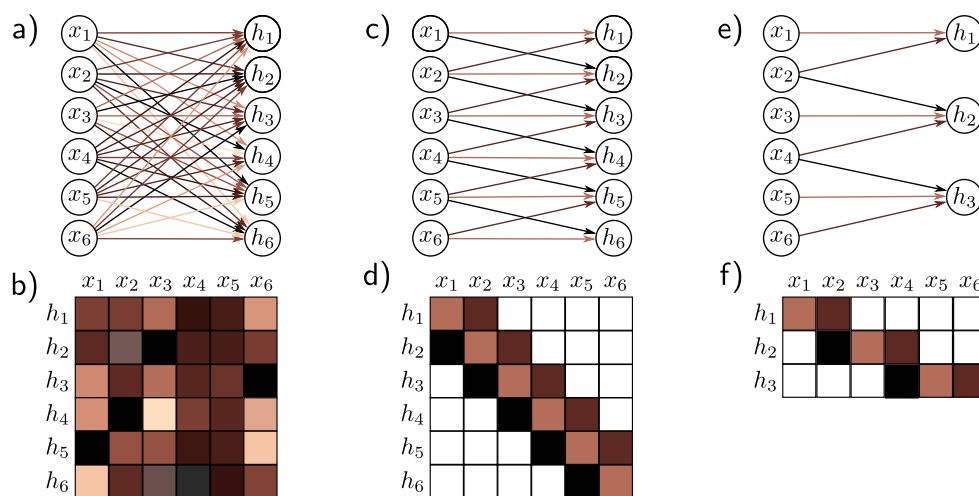


Figura 3: Fully connected vs. convolutional layers.

One possible to minimize the number of parameters is to consider local receptive fields (LRF).

Definició 8.11 (Local receptive fields (LRF)). The receptive field of a hidden unit in the network is the region of the original input that feeds into it. Moreover, a local receptive field is a standard MLP with units that have a local view of the image: its input is a submatrix $M \times M$ of the original $N \times N$, where $M < N$.

Exemple 8.12. Suppose an MNIST image size of 28×28 pixels, hence 784 dimensions (neurons). A FC layer with 100 units has $(784 \cdot 100 + 100)$ parameters, in total 78500 parameters.

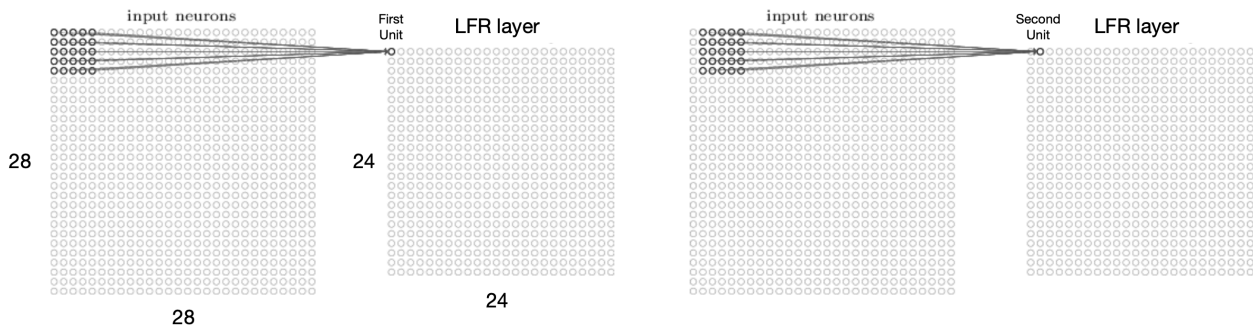


Figura 4: MNIST image convolution with LFR Layer.

We can conclude that a LRF network can be seen as a *fully connected layer where a large subset of weights are 0*.

Observació 8.13. When we evaluate the output at every position, we term this a stride of one. We have shown the local receptive field being moved by one pixel at a time, but this can be redundant in the case of natural images. In fact, sometimes a different **stride** length is used. For instance, we might move the local receptive field 2 pixels to the right (or down), in which case we'd say a *stride length of 2 is used*. This also reduces by 4 the number of units of the first layer and consequently the number of parameters.

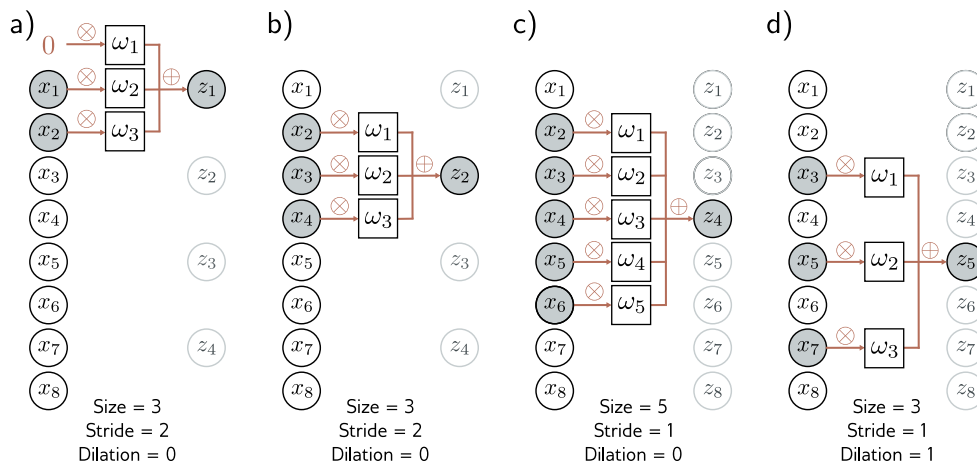
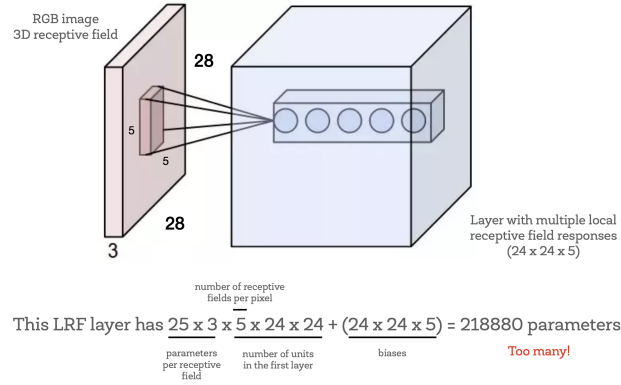


Figura 5: Stride, kernel size, and dilation. a) With a stride of two, we evaluate the kernel at every other position, so the first output z_1 is computed from a weighted sum centered at x_1 , and b) the second output z_2 is computed from a weighted sum centered at x_3 and so on. c) The kernel size can also be changed. With a kernel size of five, we take a weighted sum of the nearest five inputs.

To have more capacity, we could compute several local receptive fields per pixel and also to consider 3D local receptive fields for color images:



But in natural images, an object may appear at any image coordinates! Object detection must be invariant to translations in the image plane. In this case LRF are not suited for this task because in order to learn a detector we should provide the model with a lot of examples in order to consider all possible object localizations!

8.14 (Convolutions). An $n \times n$ (in this case, 5×5) array of shared weights (or *convolutional kernel*) is applied to a **2D input** comprising of elements x_{ij} and computes a first hidden layer of elements h_{ij} as:

$$h_{ij} = \sigma \left(\beta + \sum_{l=0}^4 \sum_{m=0}^4 w_{l,m} x_{i+l, j+m} \right),$$

where $w_{l,m}$ are the entries of the convolutional kernel, σ is the activation function and β is the shared value for the bias

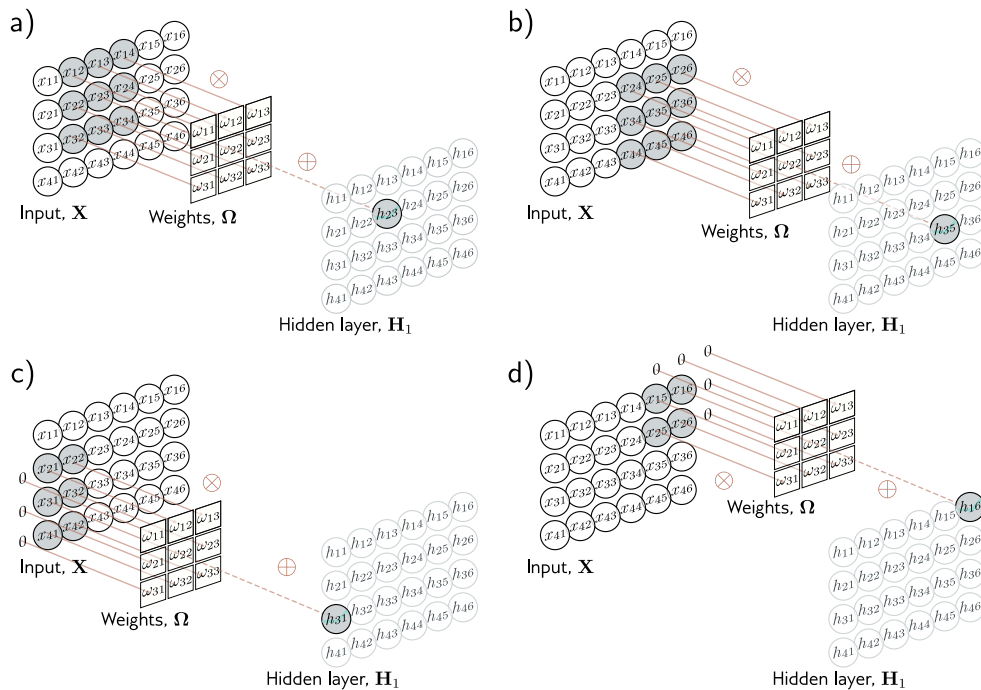


Figure 6: 2D convolutional layer.

This is simply a weighted sum over a square 5×5 input region. The kernel is translated both horizontally and vertically across the 2D input to create an output at each position.

Definició 8.15 (Convolution). The term convolution refers to the mathematical combination of two functions to produce a third function. It merges two sets of information.

8.16 (Padding). For a gray scale $(n \times n)$ image and $(f \times f)$ filter/kernel, the dimensions of the image resulting from a convolution operation is $(n - f + 1) \times (n - f + 1)$. Thus, the image shrinks every time a convolution operation is performed. This places an upper limit to the number of times such an operation could be performed (due to *stacking*) before the image reduces to nothing thereby precluding us from building deeper networks.

To overcome these problems, we use *padding*. **Padding** is simply a process of adding «dimensions» to our input images so as to avoid the problems mentioned above. There are two common approaches.

1. The first is to pad the edges of the inputs with new values and proceed as usual. *Zero-padding* assumes the input is zero outside its valid range, *replication-padding* replicates the edges' values.
2. The second approach is to discard the output positions where the kernel exceeds the range of input positions.

The weights of the first layer of a CNN can be visualized! They are $n \times n \times 3$ (color images) or $n \times n \times 1$ (BW images) tensors.

Definició 8.17 (Pooling). In addition to the convolutional layers just described, convolutional neural networks also contain pooling layers. Pooling layers are usually used immediately after convolutional layers. What the pooling layers do is simplify the information in the output from the convolutional layer.

Example 8.18 (Max-pooling). As a concrete example, one common procedure for pooling is known as max-pooling. In max-pooling, a pooling unit simply outputs the maximum activation in the 2×2 input region. Note that if we have 24×24 neurons output from the convolutional layer, after pooling we will have 12×12 neurons.

We can now put all these ideas together to form a complete 1-layer convolutional neural network:

8.19 (MNIST 1-layer convolutional neural network). The network begins with 28×28 input neurons, which are used to encode the pixel intensities for the MNIST image. This is then followed by a convolutional layer using a 5×5 local receptive field and 3 feature maps.

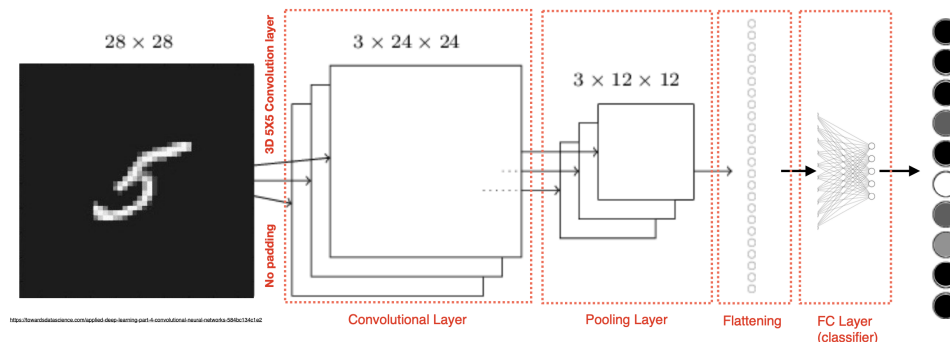


Figura 7: 1-layer convolutional neural network.

The result is a layer of $3 \times 24 \times 24$ hidden feature neurons. The next step is a max-pooling layer, applied to 2×2 regions, across each of the 3 feature maps. The result is a layer of $3 \times 12 \times 12$ hidden feature neurons. Finally, there is a fully connected layer with $3 \times 12 \times 12 \times 10 + 10$ parameters.

The stacking of convolutional layers allows a hierarchical decomposition of the input.

9

ASPECTES ÈTICS

La ciència de dades/IA pot ser tant beneficiosa com perjudicial (tenir efectes no desitjats negatius) per a les persones i/o la societat en general. La ciència de dades és l'estudi d'extreure valor de les dades, valor en forma de coneixements o conclusions.

- Coneixements:
 1. Una hipòtesi, comprovable amb més dades;
 2. Una relació plausible entre variables d'interès, descoberta examinant les dades i les implicacions de diferents escenaris;
 3. Un *eureka* que prové d'una estadística succinta o d'un gràfic visual adequat, etc.
- Conclusions:
 1. Predicció d'una conseqüència;
 2. Recomanació d'una acció útil;
 3. Clúster que agrupa elements semblants;
 4. Classificació que etiqueta elements en agrupacions;
 5. Transformació que converteix les dades a una forma més útil;
 6. Optimització que trasllada un sistema a un millor estat.

Les idees i les conclusions sovint sorgeixen de models, que són abstraccions del món real. Els models que generen aquestes conclusions poden ser de caixa clara o caixa negra. La lògica d'un model de caixa clara està disponible per a la inspecció d'altres persones, mentre que la lògica d'un model de caixa negra no. El terme «caixa opaca» també s'aplica a un model el funcionament del qual no és comprensible.

Els augments d'impacte inevitablement donen lloc a conseqüències imprevistes. Alguns usos desafien les nostres normes sobre com utilitzar la tecnologia d'una manera coherent amb els nostres valors. Molts estudiosos, educadors i empreses tecnològiques es refereixen a això com a reptes ètics.

La tecnologia i els seus impactes positius i negatius sovint es distribueixen de manera desigual entre individus i grups. Aquestes tecnologies poden perpetuar les desigualtats existents i fins i tot augmentar-les. *Exemple: Aplicacions per selecció i contractació de personal. Hereten biaixos històrics no desitjats i els poden amplificar per la seva capacitat d'escala.*

Les preocupacions ètiques de la DS/AI es poden dividir en funció del seu abast temporal i són estudiades per comunitats acadèmiques diferents:

1. Curt termini/organització: quin és l'impacte de [privadesa, transparència, equitat] a la meua aplicació?
2. Mitjà termini/societat: Com canviarà l'ús [ús militar, atenció mèdica, justícia, educació] d'aquestes aplicacions la manera com ens organitzem com a societat?
3. A llarg termini/humans: quins són els objectius ètics a llarg termini d'aquestes tecnologies? Humanisme, transhumanisme, posthumanisme?

9.1 | JUSTÍCIA

Anem a centrar-nos en una de les preocupacions ètiques més importants: la justícia. Suposem un sistema basat en dades (DS/IA) que pren decisions sobre aspectes importants en la vida d'una persona. Què vol dir que aquest sistema és «just»?

9.1 (Biaix). La definició comuna de biaix (estadístic, de dades) és que «les dades disponibles no són representatives de la població o del fenomen d'estudi». Però el biaix també pot denotar que «les dades inclouen contingut que pot contenir biaix contra grups específics de persones».

El biaix algorítmic descriu «la desviació sistemàtica en els **resultats, rendiment o impacte** d'un algorisme de decisió en relació amb alguna norma o estàndard».

Exemple 9.2 (Hidden bias). Moltes universitats utilitzen dades (*observacions*) d'estudiants anteriors per crear models per predir l'èxit (*resultats*) dels estudiants, on aquests models poden donar suport a canvis informats en polítiques (*impacte*) i pràctiques.

L'article *Hidden Bias* explora com els models d'aprenentatge automàtic poden incorporar biaixos presents en les dades del món real i com la manipulació de les variables d'entrada pot afectar aquests biaixos. A continuació, es resumeixen els escenaris presentats, destacant el predictor $\bar{y}$ utilitzat en cada cas.

1. **Escenari 1: Predicció simple basada en el GPA de secundària**
 - **Predictor $\bar{y}$:** L'esperança del GPA universitari es basa directament en el GPA de secundària dels estudiants.
 - **Observació:** Aquest mètode tendeix a sobreestimar les qualificacions universitàries, ja que els estudiants solen tenir millors notes a secundària que a la universitat.
2. **Escenari 2: Model d'aprenentatge automàtic amb més informació**

- **Predictor $\hat{y}$:** L'esperança del GPA universitari es calcula a partir de múltiples variables d'entrada dels estudiants, com ara el GPA de secundària, resultats de proves estandaritzades, activitats extracurriculars, etc.
 - **Observació:** El model ajusta les prediccions tenint en compte patrons en les dades, millorant l'exactitud en comparació amb l'enfocament simple.
3. **Escenari 3:** *Exclusió de l'atribut de gènere*
- **Predictor $\hat{y}$:** L'esperança del GPA universitari es calcula sense incloure el gènere com a variable d'entrada.
 - **Observació:** Tot i amagar el gènere, el model pot inferir proxies a partir d'altres variables, mantenint el biaix en les prediccions.
4. **Escenari 4:** *Inclusió de l'atribut de gènere*
- **Predictor $\hat{y}$:** L'esperança del GPA universitari es calcula incloent explícitament el gènere com a variable d'entrada.
 - **Observació:** En alguns casos, proporcionar aquesta informació pot permetre al model ajustar les prediccions per corregir biaixos històrics, fent-les més justes i precises.
5. **Escenari 5:** *Model amb recomanació de l'entrevistador i sense informació d'ingressos*
- **Predictor $\hat{y}$:** L'esperança del GPA universitari es calcula basant-se en la recomanació de l'entrevistador, sense informació sobre els ingressos familiars.
 - **Observació:** El biaix de l'entrevistador pot influir negativament en les prediccions del model.
6. **Escenari 6:** *Model amb recomanació de l'entrevistador i informació d'ingressos*
- **Predictor $\hat{y}$:** L'esperança del GPA universitari es calcula basant-se en la recomanació de l'entrevistador i incloent els ingressos familiars com a variable d'entrada.
 - **Observació:** El model pot corregir el biaix de l'entrevistador, millorant l'equitat i l'exactitud de les prediccions.

La gestió adequada dels atributs protegits en els models d'aprenentatge automàtic és complexa i depèn del context. En alguns casos, incloure aquests atributs pot ajudar a corregir biaixos, mentre que en altres pot perpetuar-los. És essencial analitzar cada situació per determinar l'enfocament més adequat per promoure l'equitat en les prediccions.

9.3 (Tipus de biaixos).

1. **Biaix estructural.** El *biaix estructural* fa referència als patrons i pràctiques socials o institucionals que afavoreixen uns grups i en desavantatgen d'altres basant-se en la identitat.
2. **Biaix de representació.** El *biaix de representació* es produeix quan les dades utilitzades no representen adequadament la població.
3. **Biaix de mesura.** El *biaix de mesura* apareix quan les característiques utilitzades no representen correctament el fenomen que s'està estudiant.
4. **Biaix d'agrupació.** El *biaix d'agrupació* es dona quan els grups es combinen inadequadament, cosa que pot resultar en un model que no funciona bé per a cap grup. Un exemple d'això és la *paradoxa de*

Simpson.

5. **Biaix inductiu.** El *biaix inductiu* descriu la tendència d'un sistema a preferir un conjunt determinat de generalitzacions per sobre d'altres que són igualment consistents amb les dades observades.
6. **Biaix d'avaluació.** El *biaix d'avaluació* es produeix quan, en avaluar un model, les mètriques calculades a partir d'un conjunt de proves o validació complet no sempre proporcionen una imatge precisa del funcionament del model.
7. **Biaix de desplegament.** El *biaix de desplegament* es refereix, en general, a qualsevol biaix que apareix durant el desplegament d'un sistema, on aquest s'utilitza o s'interpreta de maneres inapropiades, potser no previstes pels dissenyadors o desenvolupadors.
8. **Biaix de retroalimentació.** Aquest tipus de biaix es produeix quan el mateix model influeix en la generació de dades que s'utilitzen per entrenar-lo.

Sota els sistemes legals més avançats, tothom està protegit de comportaments il·lícits (discriminació) quan la causa d'aquest comportament és que tenen o es percep que tenen una «característica protegida» o estan associats amb algú que té una característica protegida (edat, discapacitat, gènere...).

Definició 9.4 (Equitat algorísmica). La justícia/equitat algorísmica és el camp d'investigació dirigit a comprendre i corregir biaixos no desitjats.

- Investigar les causes del biaix en dades i algorismes.
- Definició i aplicació de mesures d'equitat.
- Desenvolupament de metodologies de recollida i modelització de dades destinades a crear algorismes justos.

9.2 | MESURES D'EQUITAT

9.5. Podem distingir entre dos enfocaments per formalitzar l'equitat:

1. Les definicions d'equitat individual es basen en la premissa que les entitats similars han de ser tractades de manera similar.
2. Les definicions d'equitat del grup es basen en la definició d'entitats del grup i demanen que tots els grups siguin tractats de manera similar.

Per operar ambdós enfocaments de l'equitat, hem de definir la similitud per a l'entrada i la sortida d'un algorisme. Per a l'equitat del grup, el repte rau a determinar com dividir les entitats en grups.

Una manera de formular l'equitat individual és la basada en la **distància**. Donada una mesura de distància d entre dues entitats i una mesura de distància D entre les sortides d'un algorisme, ens agradaria que la distància entre la sortida de l'algorisme per a dues entitats fos petita, quan les entitats són semblants.

Exemple 9.6. Per simplificar, suposem dos grups, és a dir, el grup protegit G^+ (p.e. dones) i el grup no protegit G^- (o, privilegiat) (p.e. homes) i un classificador binari. Començarem presentant els enfocaments estadístics utilitzats habitualment en la classificació. Suposem això: Y és la sortida esperada i $\hat{Y}$ la predicció

d'un classificador. Sigui $\mathbf{1}$ la classe positiva que condueix a una decisió favorable (per exemple, algú que rep un préstec o que sigui admès en una escola competitiva).

Els enfocaments estadístics de l'equitat del grup es poden distingir en:

1. Els que utilitzen només estadístics de la sortida de l'algorisme (**independència**),
2. Els que usen estadístics que inclouen tant la sortida de l'algorisme com la sortida esperada (**separació**).

El criteri d'independència:

- *Compara* (ràtio o diferència) la probabilitat $P(\hat{Y} = \mathbf{1} | X \in G^+)$ que una entitat X rebi una decisió favorable quan pertany al grup protegit *amb* la probabilitat que rebi una decisió favorable quan pertany al grup no protegit, $P(\hat{Y} = \mathbf{1} | X \in G^-)$. Quan les probabilitats d'un resultat favorable són iguals per als dos grups, tenim un tipus especial d'equitat anomenada **paritat demogràfica o estadística**:

$$P(\hat{Y} = \mathbf{1} | X \in G^+) \sim P(\hat{Y} = \mathbf{1} | X \in G^-).$$

- *Aquest mètode ignora «els mèrits» de X* . Per exemple, suposem que la tasca de classificació és aconseguir o no una feina i el grup protegit G^+ està basada en el gènere. La paritat estadística demana una ràtio específica de dones en la classe positiva, fins i tot quan no hi ha tantes dones a l'entrada que estiguin ben qualificades per al treball.

El criteri de precisió:

- L'equitat basada en la precisió (que correspon als mètodes que usen tant la sortida $\hat{Y}$ de l'algorisme com la sortida esperada Y) garanteix que diversos tipus d'errors de classificació (per exemple, *true positives* i *false negatives*) són iguals entre els grups. En funció del tipus d'errors de classificació considerats, el tipus d'equitat aconseguït pren diferents noms.

		True condition			
Total population		Condition positive	Condition negative	Prevalence $= \frac{\Sigma \text{Condition positive}}{\Sigma \text{Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{True positive} + \Sigma \text{True negative}}{\Sigma \text{Total population}}$
Predicted condition	Predicted condition positive	True positive	False positive, Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{True positive}}{\Sigma \text{Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{False positive}}{\Sigma \text{Predicted condition positive}}$
	Predicted condition negative	False negative, Type II error	True negative	False omission rate (FOR) = $\frac{\Sigma \text{False negative}}{\Sigma \text{Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{True negative}}{\Sigma \text{Predicted condition negative}}$
$\hat{Y} = 1 \quad Y = 1$ True positive rate, recall $\hat{Y} = 0 \quad Y = 1$ False negative rate $\hat{Y} = 1 \quad Y = 0$ False positive rate $\hat{Y} = 0 \quad Y = 0$ True negative rate		True positive rate (TPR), Recall, Sensitivity, probability of detection, Power $= \frac{\Sigma \text{True positive}}{\Sigma \text{Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm $= \frac{\Sigma \text{False positive}}{\Sigma \text{Condition negative}}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) $= \frac{\text{LR+}}{\text{LR-}}$ $F_1 \text{ score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
		False negative rate (FNR), Miss rate $= \frac{\Sigma \text{False negative}}{\Sigma \text{Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) $= \frac{\Sigma \text{True negative}}{\Sigma \text{Condition negative}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$	

El cas en què demanem la mateixa taxa de True Positives s'anomena separació:

$$P(\hat{Y} = 1 | Y = 1 \cap X \in G^+) = \frac{P(\hat{Y} = 1 \cap Y = 1 \cap X \in G^+)}{P(Y = 1 \cap X \in G^+)} = P(\hat{Y} = 1 | Y = 1 \cap X \in G^-).$$

Comparant aquesta mètrica amb la paritat estadística, de nou els membres dels dos grups tenen les mateixes possibilitats d'obtenir el resultat favorable, però només quan aquests membres compleixin els requisits.

Es pot demostrar que si suposem que el problema de decisió és binari, la independència i la separació no es poden mantenir al mateix temps.

És impossible satisfer totes les definicions d'equitat del grup, el que significa que els científics de dades han d'escollir-ne una per fer referència quan inicien una anàlisi d'equitat.

La incompatibilitat de les mètriques d'equitat no implica que els esforços per tenir sistemes equitables siguin infructuosos. En canvi, suggereix que l'equitat s'ha de definir contextualment per a un problema determinat, amb l'objectiu de prevenir danys específics dels seus casos d'ús.

A NOTEBOOKS

A.1 | SINGLE-LAYER PERCEPTRONS

A simple model we could build is a single layer perceptron.

Definició A.1 (Perceptron). A perceptron uses a weighted linear combination of the inputs to return a prediction score. If the prediction score exceeds a selected threshold, the perceptron predicts True. Otherwise it predicts False. More formally,

$$f(x) = \begin{cases} 1 & \text{if } w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 > b \\ 0 & \text{otherwise.} \end{cases} \xrightarrow{\hat{y} = \mathbf{w} \cdot \mathbf{x} + b} f(x) = \begin{cases} 1 & \text{if } \hat{y} > 0 \\ 0 & \text{otherwise.} \end{cases}$$

Here $\hat{y}$ is our prediction score.

Exemple A.2. For our example, suppose we build the following perceptron:

$$\hat{y} = -0.0019x_1 + (-0.0016x_2) + 0.0020x_3 + 0.0023x_4 + 0.0003.$$

Here's how the perceptron would perform on some of our training images.

$$\begin{aligned} x = [47, 250, 8, 88] &\implies \hat{y} = -0.27 < 0 & x = [181, 5, 157, 152] &\implies \hat{y} = 0.31 > 0 \\ x = [115, 138, 80, 88] &\implies \hat{y} = -0.08 < 0 & x = [0, 212, 169, 167] &\implies \hat{y} = 0.38 > 0 \\ x = [225, 13, 210, 144] &\implies \hat{y} = 0.30 > 0 & x = [12, 61, 158, 248] &\implies \hat{y} = 0.77 > 0 \end{aligned}$$

This would certainly be better than randomly guessing and it makes some logical sense: all the stairs patterns have darkly shaded pixels in the bottom row which supports the positive coefficients for x_3 and x_4 . Nonetheless, there are some glaring problems with this model.

1. The model outputs a real number whose value correlates with the concept of likelihood (higher values imply a greater probability the image represents stairs) but there's no basis to interpret the values as probabilities, specially since they can be outside the range $[0, 1]$.
2. The model can't capture the non-linear relationship between the variables and the target.

Consider the following hypothetical scenarios. First case, start with an image $x = [100, 0, 0, 125]$ and increase x_3 from 0 to 60. Second case, start with an image $x = [100, 0, 60, 125]$ and increase x_3 from 60 to 120. Intuitively, first case should have a much larger increase in $\hat{y}$ than second case. However, since our perceptron model is a linear equation, the equivalent +60 change in x_3 resulted in an equivalent +0.12 change in $\hat{y}$, for *both* cases.

These problems can be fixed by wrapping our perceptron within a sigmoid function (and subsequently choosing different weights).

Definició A.3 (Sigmoid function). The sigmoid function is an S shaped curve bounded on the vertical axis between 0 and 1, and is thus frequently used to model the probability of a binary event. Logistic functions are a special case:

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}, \text{ im}(\text{sigmoid}(z)) \in [0, 1].$$

Following this idea, we can update our model with the following picture and equation.

$$z = \mathbf{w} \cdot \mathbf{x} + b = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b, \hat{y} = \text{sigmoid}(z) \in [0, 1].$$

Example A.4 (Continuation, A.2). We can simplify notation by absorbing b in $\mathbf{w}$. This can be done by extending all examples with an extra dimension with value 1: $x = [100, 0, 0, 125] \rightarrow x = [100, 0, 0, 125, 1]$. Then, $\mathbf{w}$ will have an extra dimensional and its last value will represent b .

Now $\hat{y} \in [0, 1]$, and can now interpret it as a probability and we must update our decision rule accordingly. Using that $z > 0 \iff e^{-z} < 1 \iff 1 + e^{-z} < 2 \iff \frac{1}{2} < \frac{1}{1+e^{-z}}$:

$$z > 0 \iff \text{sigmoid}(z) = \frac{1}{1 + e^{-z}} > \frac{1}{1 + 1} = \frac{1}{2} \implies f(x) = \begin{cases} 1 & \text{if } \hat{y} > 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

Consider the hypothetical scenarios from A.2. Notice how the curvature of the sigmoid function causes first case to increase rapidly as $z = \mathbf{w} \cdot \mathbf{x}$ increases, but the pace slows down as z continues to increase. This aligns with our intuition that first case should reflect a greater increase in the likelihood of stairs versus second case.

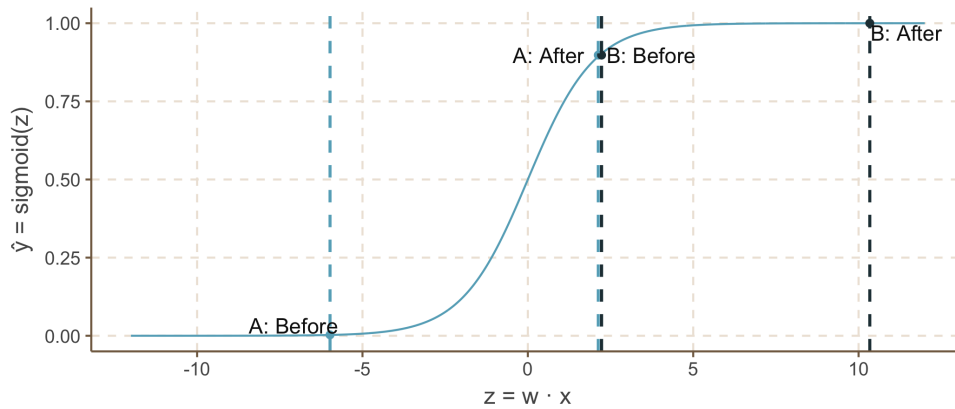


Figura 8: Sigmoid for cases $\mathcal{A}$ (first) and $\mathcal{B}$ (second).

Observació A.5.

1. $\hat{y}$ has a monotonic relationship with each variable. What if we want to identify lightly shaded stairs?
2. The model does not account for variable interaction. Assume the bottom row of an image is black. If the top left pixel is white, darkening the top right pixel should increase the probability of stairs. If the top left pixel is black, darkening the top right pixel should decrease the probability of stairs. In other words, increasing x_3 should potentially increase or decrease $\hat{y}$ depending on the values of the other variables. Our current model has no way of achieving this.
3. **Perceptron cannot do XOR.** XOR is defined as follows:

input	output	input	output
(0, 0)	0	(1, 0)	1
(0, 1)	1	(1, 1)	0

It is not difficult to see that XOR is the usual addition in abelian group $\mathbb{Z}/2\mathbb{Z}^4$. XOR single layer perceptron would consist of the following inequalities which yield a contradiction:

$$\begin{cases} 0 \cdot w_1 + 0 \cdot w_2 < t \\ 1 \cdot w_1 + 1 \cdot w_2 < t \\ 1 \cdot w_1 + 0 \cdot w_2 \geq t \\ 0 \cdot w_1 + 1 \cdot w_2 \geq t \end{cases} \implies \begin{cases} 0 < t \\ w_1 + w_2 < t \\ w_1 \geq t > 0 \\ w_2 \geq t > 0 \end{cases} \implies \begin{cases} w_1 + w_2 < t \\ w_1 + w_2 \geq t > 0 \end{cases}$$

As we have seen, «single-layer» perceptron can't implement XOR. The reason is because the classes in XOR are not linearly separable. You cannot draw a straight line to separate the points (0, 0), (1, 1) from the points (0, 1), (1, 0).

⁴ We could finish here saying that activation functions must be monotonic and modulo is not a monotonic function, but we will give a proof by contradiction.

A.2 | MULTI-LAYER PERCEPTRON

We can solve both of the above issues by adding an **extra layer** to our perceptron model. We'll construct a number of base models like the one above, but then we'll feed the output of each base model as input into another perceptron. This model is in fact a vanilla neural network.

In our examples we used a sigmoid activation function. However, we could use other activation functions. $\tanh$ and *relu* are common choices today because of training issues when learning the optimal weights. The **activation function must be non-linear**, otherwise the neural network would simplify to an equivalent single layer perceptron.

A.3 | MULTICLASS CLASSIFICATION

We can easily extend our model to work for multiclass classification by using multiple nodes in the final output layer. The idea here is that each output node corresponds to one of the C classes we are trying to predict. Instead of squashing the output with the sigmoid function which maps an element in $\mathbb{R}$ to an element in $[0, 1]$, we can use the softmax function which maps a vector in $\mathbb{R}^n$ to a vector in $\mathbb{R}^n$ such that the resulting vector elements sum to 1.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

In other words, we can design the network such that it outputs the vector:

$$[\text{prob}(\text{class}_1), \text{prob}(\text{class}_2), \dots, \text{prob}(\text{class}_C)].$$

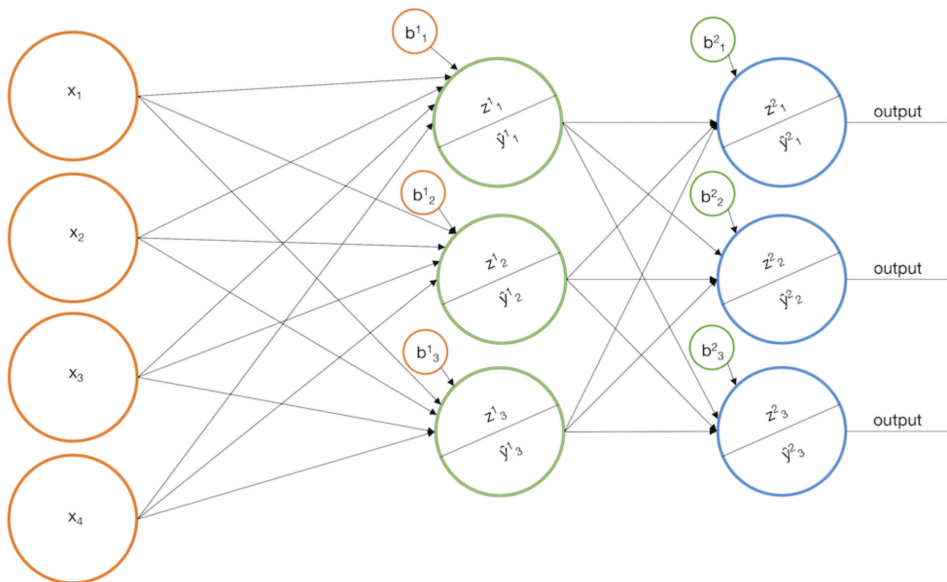


Figura 9: Multiclass classification.

A.4 | DEEP LEARNING AND BACKPROPAGATION

Can we extend our vanilla neural network so that its output layer is fed into a 4th layer (and then a 5th, and 6th, etc.)? Yes. This is what's commonly referred to as «deep learning». In practice it can be very effective. However, it's worth noting that, in theory, any network you build with more than one hidden layer can be mimicked by a network with only one hidden layer (Universal Approximation Theorem). The reasons deep neural network architectures are chosen in favor of single hidden layer architectures are:

1. They converge to a solution during the fitting procedure much more often and much more faster.
2. They use less parameters.

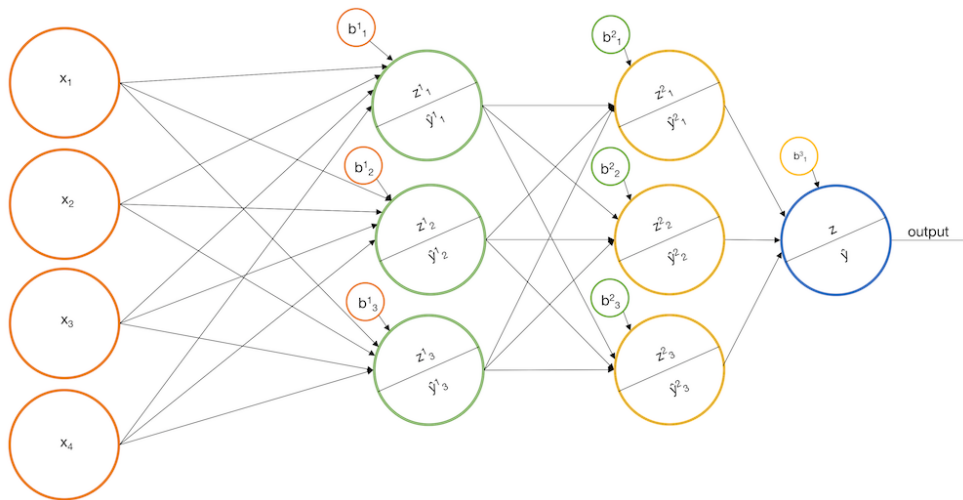


Figura 10: Using more than two layers.

So far we've discussed how neural networks could work effectively, but we haven't discussed how to fit a neural network to labeled training samples. *How can we choose the best weights for a network, given some labeled training samples?* **Stochastic gradient descent** is the common answer.

1. Start with some labeled training data.
2. Choose a differentiable loss function to minimize, $L(\hat{Y}, Y)$.
3. Choose a network structure. Specifically determine how many layers and how many nodes in each layer.
4. Initialize the network's weights randomly.
5. Run a training data point through the network to generate a prediction. Measure the overall error according to the loss function, $L(\hat{Y}, Y)$. (This is called forward propagation).
6. Determine how much the current loss will change with respect to a small change in each of the weights.
7. In other words, calculate the gradient of L with respect to every weight in the network. (This is called backward propagation).
8. Take a small «step» in the direction of the negative gradient.
9. Repeat this process (from step 5) a fixed number of epochs (until the loss converges).